

OpsPulse Incident Runbook: Kubernetes Ingress 504 Timeouts

1. Incident Description & Scope

Symptoms: Ingress-Nginx returns HTTP 504 Gateway Timeout to external clients.

Upstream service pods fail to respond within the configured proxy-read-timeout.

Log Pattern: [error] upstream timed out (110: Connection timed out) while reading response header.

Affected Components: Ingress Controller, API Gateway, Upstream Microservices, Service Mesh.

2. Root Cause Diagnostics Protocol

Step 1: Check Ingress Controller error logs for timeout spikes and upstream IP addresses:

```
kubectl logs -n ingress-nginx -l app.kubernetes.io/name=ingress-nginx --tail=200 | grep 504
```

Step 2: Inspect upstream service endpoints and verify pod health:

```
kubectl get endpoints order-service -n production
```

Step 3: Check CPU and memory saturation on worker pods:

```
kubectl top pods -n production -l app=order-service
```

Step 4: Check if upstream pods are thread starved or waiting on downstream database pool.

3. Emergency Remediation & Fix Actions

Action A (Immediate Scale Out): Increase replicas to distribute request load immediately:

```
kubectl scale deployment order-service -n production --replicas=10
```

Action B (Temporary Timeout Increase): Annotate ingress with extended timeout during incident:

```
nginx.ingress.kubernetes.io/proxy-read-timeout: 120
```

Action C (Graceful Rollout): Restart blocked pods exhibiting deadlocks or high garbage collection:

```
kubectl rollout restart deployment/order-service -n production
```

OpsPulse Incident Runbook: PostgreSQL Connection Pool Saturation

1. Incident Description & Scope

Symptoms: Microservices throw database connection errors. API latency spikes to 10+ seconds.

Log Pattern: FATAL: remaining connection slots are reserved for non-replication superuser connections

Secondary Pattern: org.postgresql.util.PSQLException: FATAL: sorry, too many clients already

Affected Components: PostgreSQL cluster, PgBouncer pooler, Auth Service, Checkout Service.

2. Root Cause Diagnostics Protocol

Step 1: Inspect connection counts by application state in PostgreSQL:

```
SELECT count(*), state, username FROM pg_stat_activity GROUP BY state, username;
```

Step 2: Detect long-running queries holding locks and starving connection pools:

```
SELECT pid, client_addr, now() - query_start AS duration, state, query FROM pg_stat_activity  
WHERE state != idle AND now() - query_start > interval 30 seconds ORDER BY duration DESC LIMIT 10;
```

Step 3: Identify blocking locks and blocked transactions:

```
SELECT blocked_locks.pid AS blocked_pid, blocking_locks.pid AS blocking_pid, blocking_activity.query  
FROM pg_catalog.pg_locks blocked_locks  
JOIN pg_catalog.pg_locks blocking_locks ON blocking_locks.locktype = blocked_locks.locktype;
```

3. Emergency Remediation & Fix Actions

Action A (Terminate Runaway Transactions): Kill blocking PID immediately:

```
SELECT pg_terminate_backend(blocking_pid) FROM pg_stat_activity WHERE pid = <blocking_pid>;
```

Action B (Enforce Idle Transaction Timeout): Disconnect leaked idle connections automatically:

```
ALTER DATABASE production_db SET idle_in_transaction_session_timeout = 20000;
```

Action C (PgBouncer Configuration Tuning):

Ensure max_client_conn is 1000 and default_pool_size is adjusted to avoid backend saturation.